

\$\$\$ controle.alocacao_equipamentos

```
from src.util.gerais import imprimir_objetos, ordenar_objetos_por_um_atributo

from src.util.data import Data

from src.entidades.empreiteiro import Empreiteiro, get_empreiteiros,
inserir_empreiteiro

from src.entidades.equipamento import Equipamento, get_equipamentos,
inserir_equipamento

from src.entidades.obra import Obra, get_obras, inserir_obra


def cadastrar_equipamentos():

    inserir_equipamento(Equipamento("Furadeira", "Ferramenta Elétrica", 20, False))
    inserir_equipamento(Equipamento("Lixadeira", "Ferramenta Elétrica", 18, True))
    inserir_equipamento(Equipamento("Betoneira", "Construção", 320, False))
    inserir_equipamento(Equipamento("Andaime", "Construção", 200, True))
    inserir_equipamento(Equipamento("Capacete de proteção", "Segurança", 1, False))
    inserir_equipamento(Equipamento("Cinto de segurança", "Segurança", 3, False))
    inserir_equipamento(Equipamento("Carrinho de mão", "Transporte", 15, True))
    inserir_equipamento(Equipamento("Empilhadeira", "Transporte", 3500, True))
    inserir_equipamento(Equipamento("Trena", "Medição", 0.5, False))
    inserir_equipamento(Equipamento("Nível a laser", "Medição", 2, True))
    inserir_equipamento(Equipamento("Serra circular", "Corte", 12, False))
    inserir_equipamento(Equipamento("Esmerilhadeira", "Corte", 8, True))
    inserir_equipamento(Equipamento("Guincho", "Elevação", 500, True))
    inserir_equipamento(Equipamento("Talha manual", "Elevação", 50, False))


def cadastrar_empreiteiros():

    inserir_empreiteiro(Empreiteiro("Construções Dourados Ltda.", "(67) 9999-8888",
```

```

        "contato.dourados@gmail.com", "Rua dos Pedreiros, 123"))

    inserir_empiteiro(Empiteiro("Serviços de Telecomunicação", "(67) 8888-7777",
        "tele.servicos@gmail.com", "Avenida da Obra, 456"))

    inserir_empiteiro(Empiteiro("Construção Civil", "(67) 7777-6666",
        "contato.g3@gmail.com", "Rua da Inovação, 789"))

def cadastrar_obras():

    empreiteiros = get_empiteiros()
    equipamentos = get_equipamentos()

    obra1 = Obra(837, "Construção do Edifício Sol", Data(2, 3, 2023),
        empreiteiro=empreiteiros["Construções Dourados Ltda."])
    obra1.inserir_equipamentos(["Betoneira", "Andaime", "Guincho", "Furadeira"])
    inserir_obra(obra1)

    obra2 = Obra(524, "Reforma da Escola Municipal", Data(15, 4, 2023),
        empreiteiro=empreiteiros["Serviços de Telecomunicação"])
    obra2.inserir_equipamentos(["Lixadeira", "Capacete de proteção", "Cinto de
segurança", "Nível a laser"])
    inserir_obra(obra2)

    obra3 = Obra(796, "Shopping novo da cidade", Data(16, 9, 2024),
        empreiteiro=empreiteiros["Construção Civil"])
    obra3.inserir_equipamentos(["Carrinho de mão", "Trena", "Guincho"])
    inserir_obra(obra3)

if __name__ == '__main__':
    cadastrar_equipamentos()

```

```
cadastrar_empregados()
```

```
cadastrar_obras()
```

```
imprimir_objetos('Obra : id, descrição, datas, empregado', get_obras().values())
```

```
imprimir_objetos('Empregados: nome, telefone, email, endereço',  
get_empregados().values())
```

```
for obra in get_obras().values():
```

```
    print('\n\n==== ' + obra.descrição + ' ===')
```

```
    equipamentos_obra = obra.equipamentos.values()
```

```
    imprimir_objetos('Equipamento : nome, tipo, valor, status', equipamentos_obra)
```

```
    equipamentos_ordenados = ordenar_objetos_por_um_atributo(
```

```
        objetos=equipamentos_obra,
```

```
        comparador=lambda e1, e2: (e1.disponivel and not e2.disponivel)
```

```
    )
```

```
    imprimir_objetos('Equipamento : nome, tipo, valor, status -- disponíveis primeiro',  
        equipamentos_ordenados)
```

\$\$\$ entidades.empregado

```
empregados = {}
```

```
class Empregado:
```

```
    def __init__(self, nome, telefone, email, endereco):
```

```
        self.nome = nome
```

```
        self.telefone = telefone
```

```
        self.email = email
```

```
        self.endereco = endereco
```

```
    def __str__(self):
```

```
        return f'{self.nome:<30} {self.telefone:<20} {self.email:<30} {self.endereco:<25}'
```

```
def get_empregados():
```

```
    return empregados
```

```
def inserir_empregado(empregado):
```

```
    nome = empregado.nome
```

```
    if nome not in empregados.keys():
```

```
        empregados[nome] = empregado
```

```
    else:
```

```
        print(f'Empregado {nome} já tem cadastro.')
```

\$\$\$ entidades.equipamento

```
equipamentos = {}
```

```
class Equipamento:
```

```
    def __init__(self, nome, tipo, valor, disponivel):
```

```
        self.nome = nome
```

```
        self.tipo = tipo if tipo in (
```

```
            'Ferramenta Elétrica', 'Construção', 'Segurança',
```

```
            'Transporte', 'Medição', 'Corte', 'Elevação'
```

```
        ) else 'indefinido'
```

```
        self.valor = valor
```

```
        self.disponivel = disponivel
```

```
    def __str__(self):
```

```
        disponivel_str = 'disponível' if self.disponivel else '    '
```

```
        return f'{self.nome:<25} {self.tipo:<25} R$ {self.valor:<10.2f} {disponivel_str:<15}'
```

```
    def get_equipamentos():
```

```
        return equipamentos
```

```
    def inserir_equipamento(equipamento):
```

```
        equipamentos[equipamento.nome] = equipamento
```

\$\$\$ entidades.obra

```
from src.util.data import Data
```

```
from src.entidades.empreiteiro import Empreiteiro
```

```
from src.entidades.equipamento import get_equipamentos
```

```
obras = {}
```

```
class Obra:
```

```
    def __init__(self, id, descrição, data_inicio: Data, data_fim: Data = None, empreiteiro: Empreiteiro = None):
```

```
        self.id = id
```

```
        self.descrição = descrição
```

```
        self.data_inicio = data_inicio
```

```
        self.data_fim = data_fim
```

```
        self.empreiteiro = empreiteiro
```

```
        self.equipamentos = {}
```

```
    def inserir_equipamentos(self, chaves_equipamentos):
```

```
        for chave_equipamento in chaves_equipamentos:
```

```
            if chave_equipamento in get_equipamentos().keys():
```

```
                self.equipamentos[chave_equipamento] =  
get_equipamentos()[chave_equipamento]
```

```
            else:
```

```
                print(f'Equipamento {chave_equipamento} não tem cadastro.')
```

```
    def remover_equipamento(self, equipamento):
```

```
        chave = equipamento.nome
```

```
        if chave in self.equipamentos:
```

```
            del self.equipamentos[chave]
```

else:

print(f'Equipamento {chave} não está alocado na Obra {self.id}.')

def listar_equipamentos(self):

return list(self.equipamentos.values())

def __str__(self):

data_fim_str = str(self.data_fim) if self.data_fim else '---'

empreiteiro_str = self.empreiteiro.nome if self.empreiteiro else '---'

formato = '{:<12} {:<35} {:<12} {:<12} {:<25}'

return formato.format(self.id, self.descrição[:34], str(self.data_inicio), data_fim_str,
empreiteiro_str)

def get_obras():

return obras

def inserir_obra(obra):

id_obra = obra.id

if id_obra not in obras.keys():

obras[id_obra] = obra

else:

print(f'Obra {id_obra} já tem cadastro.')

\$\$\$ util.data

```
from datetime import date
```

```
class Data:
```

```
    def __init__(self, dia, mês, ano):
```

```
        self.dia = dia
```

```
        self.mês = mês
```

```
        self.ano = ano
```

```
    def __str__(self):
```

```
        data_str = "
```

```
        if self.dia < 10:
```

```
            data_str = '0' + str(self.dia)
```

```
        else:
```

```
            data_str = str(self.dia)
```

```
        if self.mês < 10:
```

```
            data_str += "/0" + str(self.mês)
```

```
        else:
```

```
            data_str += "/" + str(self.mês)
```

```
        data_str += "/" + str(self.ano)
```

```
        return data_str
```

```
    def calcular_idade(self, data_referencia=None):
```

```
        if data_referencia is None:
```

```
            dia_atual_str, mês_atual_str, ano_atual_str =  
date.today().strftime("%d/%m/%Y").split('/')
```

```
            dia_referencia, mês_referencia, ano_referencia = int(dia_atual_str),  
int(mês_atual_str), int(ano_atual_str)
```

```
        else:
```



```
        dia_referencia, mês_referencia, ano_referencia = data_referencia.dia,  
data_referencia.mês, data_referencia.ano
```

```
idade = ano_referencia - self.ano
```

```
        if mês_referencia < self.mês or (mês_referencia == self.mês and dia_referencia <  
self.dia):
```

```
            idade -= 1
```

```
        return idade
```

```
def __eq__(self, data):
```

```
        if self.dia == data.dia and self.mês == data.mês and self.ano == data.ano: return  
True
```

```
        return False
```

```
def __ne__(self, data): return not self == data
```

```
def __gt__(self, data):
```

```
    if self.ano > data.ano: return True
```

```
    elif self.ano < data.ano: return False
```

```
    if self.mês > data.mês: return True
```

```
    elif self.mês < data.mês: return False
```

```
    if self.dia > data.dia: return True
```

```
    elif self.dia < data.dia: return False
```

```
    return False
```

```
def __lt__(self, data):
```

```
    if self.ano < data.ano: return True
```

```
elif self.ano > data.ano: return False  
if self.mês < data.mês: return True  
elif self.mês > data.mês: return False  
if self.dia < data.dia: return True  
elif self.dia > data.dia: return False  
return False
```

```
def __ge__(self, data):  
    if self < data: return False  
    else: return True
```

```
def __le__(self, data):  
    if self > data: return False  
    else: return True
```

\$\$\$ util.gerais

```
def compacta_nome(nome):  
    partes_nome = nome.split(' ')  
    nome_compactado = "  
    for parte_nome in partes_nome:  
        nome_compactado += parte_nome if parte_nome not in ('de', 'da', 'das', 'do',  
'dos') else "  
    return nome_compactado  
  
def imprimir_objetos(cabeçalho, objetos):  
    print('\n' + cabeçalho)  
    for indice, objeto in enumerate(objetos):  
        formato = '{:<5} {}'  
        print(formato.format(str(indice + 1) + ' -', str(objeto)))  
  
def ordenar_objetos_por_um_atributo(objetos, comparador):  
    objetos_ordenados = []  
    for objeto_desordenado in objetos:  
        ordenou_objeto = False  
        for indice, objeto_ordenado in enumerate(objetos_ordenados):  
            if comparador(objeto_desordenado, objeto_ordenado):  
                objetos_ordenados.insert(indice, objeto_desordenado)  
                ordenou_objeto = True  
                break  
        if not ordenou_objeto:  
            objetos_ordenados.append(objeto_desordenado)  
    return objetos_ordenados
```

Dourados, 23/09/2025

Rian Ruela Ribeiro